Practical : 1	Implementation of All sorting algorithms

Aim:

To implement and analyze different sorting algorithms such as Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.

SORTING ALGORITHMS

1. Bubble Sort

1. Start.
2. Read the array and size n.
3. Compare adjacent elements.
4. If the first element is greater than the next element, swap them.
5. Repeat the comparison for the remaining elements.
6. After each pass, the largest element moves to the end.
7. Repeat until the array is sorted.
8. Stop.

2. Selection Sort

1. Start.
2. Read the array and size n.
3. Select the first unsorted position.
4. Find the minimum element in the remaining array.
5. Swap the minimum element with the current position.
6. Move to the next position.
7. Repeat until the array is sorted.
8. Stop.

3. Insertion Sort

1. Start.
2. Read the array and size n.

3. Select the next element as key.
4. Compare the key with elements in the sorted portion.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat for all elements.
8. Stop.

4. Merge Sort

1. Start.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Repeat until the complete array is sorted.
7. Stop.

5. Quick Sort

1. Start.
2. Select the last element as the pivot.
3. Partition the array around the pivot.
4. Place smaller elements before the pivot.
5. Place larger elements after the pivot.
6. Recursively sort the left and right portions.
7. Stop.

Program:-

```
#include <iostream> using

namespace std; // Function to print

array elements void printArray(int

arr[], int size) { for (int i = 0; i <

size; i++) { cout << arr[i] << " ";

} cout << endl; } // 1. Bubble Sort
```

```
void bubbleSort(int arr[], int n) {
for (int i = 0; i < n - 1; i++) { for
(int j = 0; j < n - i - 1; j++) { if
(arr[j] > arr[j + 1]) { int temp =
arr[j]; arr[j] = arr[j + 1]; arr[j +
1] = temp; }
}
} } // 2. Selection Sort void
selectionSort(int arr[], int n) { for
(int i = 0; i < n - 1; i++) { int
minIndex = i; for (int j = i + 1; j <
n; j++) { if (arr[j] <
arr[minIndex]) { minIndex = j; }
} int temp = arr[minIndex];
arr[minIndex] = arr[i]; arr[i] =
temp; } } // 3. Insertion Sort void
insertionSort(int arr[], int n) { for
(int i = 1; i < n; i++) { int key =
arr[i]; int j = i - 1; while (j >= 0
&& arr[j] > key) {
arr[j + 1] = arr[j]; j--
;
```

```
} arr[j + 1]
= key; }
}
// Merge helper function void merge(int arr[],
int left, int mid, int right) { int n1 = mid - left +
1; int n2 = right - mid; int
L[100], R[100]; for
(int i = 0; i < n1; i++) L[i]
= arr[left + i]; for (int j =
0; j < n2; j++) R[j] =
arr[mid + 1 + j]; int i = 0,
j = 0, k = left; while (i <
n1 && j < n2) { if (L[i]
<= R[j]) { arr[k] = L[i];
i++; } else { arr[k] =
R[j]; j++; } k++; } while
(i < n1) {
arr[k] = L[i];
i++; k++; }
while (j < n2) {
arr[k] = R[j];
j++; k++;
```

```
}
}
// 4. Merge Sort void mergeSort(int arr[],
int left, int right) { if (left < right) { int mid
= left + (right - left) / 2; mergeSort(arr, left,
mid); mergeSort(arr, mid + 1, right);
merge(arr, left, mid, right);
}
}
// Partition function for Quick Sort int
partition(int arr[], int low, int high) { int
pivot = arr[high]; int i = low - 1; for (int
j = low; j < high; j++) { if (arr[j] < pivot)
{ i++; int temp = arr[i]; arr[i] = arr[j];
arr[j] = temp;
} } int temp = arr[i + 1]; arr[i + 1] =
arr[high]; arr[high] = temp; return i + 1; }
// 5. Quick Sort void quickSort(int arr[], int
low, int high) { if (low < high) { int pi =
partition(arr, low, high);
```

```
quickSort(arr, low, pi - 1);

quickSort(arr, pi + 1, high);

} } int main() { int originalArr[] = {64, 34, 25, 12,

22, 11, 90}; int n = sizeof(originalArr) /

sizeof(originalArr[0]); int arr[100]; cout <<

"Original Array: "; printArray(originalArr, n);

// Bubble Sort for (int i = 0; i < n;

i++) arr[i] = originalArr[i];

bubbleSort(arr, n); cout << "\n1.

Bubble Sort Result: ";

printArray(arr, n); // Selection Sort

for (int i = 0; i < n; i++) arr[i] =

originalArr[i]; selectionSort(arr,

n); cout << "2. Selection Sort

Result: "; printArray(arr, n); //

Insertion Sort for (int i = 0; i < n;

i++) arr[i] = originalArr[i];

insertionSort(arr, n); cout << "3.

Insertion Sort Result: ";

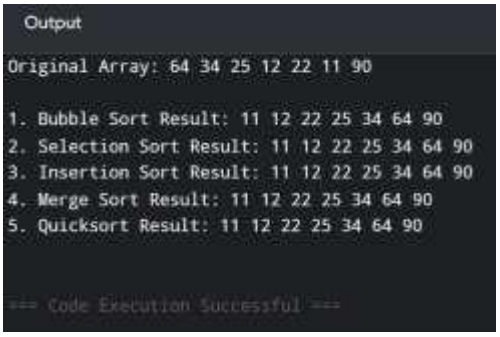
printArray(arr, n); // Merge Sort

for (int i = 0; i < n; i++) arr[i] =

originalArr[i]; mergeSort(arr, 0, n
```

```
- 1); cout << "4. Merge Sort
Result: ";
printArray(arr, n); // Quick
Sort for (int i = 0; i < n; i++)
arr[i] = originalArr[i];
quickSort(arr, 0, n - 1); cout <<
"5. Quicksort Result: ";
printArray(arr, n); return 0;
}
```

Output:



Analysis:-

The algorithms tested can be naturally grouped into two categories based on their design patterns: Quadratic-time basic sorts O(n^2)and Logarithmic-time advanced divide-andconquer sorts O(n log n)

TimeComplexity Summary

Bubble, Selection, and Insertion Sort utilize nested loop structures. In average and worst-case setups, they complete roughly n^2 computational comparison loops, giving them an O(n^2)

quadratic rate of growth. Insertion and optimized Bubble Sort perform exceptionally well O(n) if the dataset is already mostly sorted.

Merge Sort and Quicksort split the data into smaller fractions continuously. This divide-andconquer

strategy dramatically cuts down operations to an O(n log n) average rate.

Quicksort can degrade to O(n^2) if the pivot choices consistently isolate single elements

Instead of balanced splits

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	Ω(N)	Θ(N^2)	O(N^2)	O(1)
Selection Sort	Ω(N^2)	Θ(N^2)	O(N^2)	O(1)
Insertion Sort	Ω(N)	Θ(N^2)	O(N^2)	O(1)
Quick Sort	Ω(N log N)	Θ(N log N)	O(N^2)	O(N)
Merge Sort	Ω(N log N)	Θ(N log N)	O(N log N)	O(N)

SpaceComplexitySummary

a) Bubble, Selection, and Insertion Sort require O(1)space because they arrange elements completely in-place without generating extra lists.

b) Merge Sort requires a linear auxiliary footprint of O(n) memory to construct temporary arrays during active array blending operations.

c) Quicksort uses an execution stack footprint of O(log n) on average to keep track of recursive partition boundaries.

Conclusion:-

This practical highlights the direct real-world trade-off between algorithmic simplicity and scalability:

For Small Datasets: Simple algorithms like Insertion Sort and Bubble Sort are optimal because they require negligible implementation complexity and execute completely in place (O(1)auxiliary space).

For Large Datasets: Divide-and-conquer algorithms like Merge Sort and Quicksort are vastly superior because their O(n log n) runtime allows them to process massive files cleanly. Merge Sort guarantees reliable stability but requires a higher memory profile (O(n)), while Quicksort offers highly rapid in-place processing under typical runtime conditions.